This final course has offered me the opportunity to showcase what I've learned throughout the program. The artifacts that I have worked on that allowed me to create this artifact for my ePortfolio have encompassed a lot of fundamental computer science concepts. Starting with the basics of object-oriented programming: inheritance, encapsulation, abstraction, and polymorphism. This program has exposed me to linear algebra, machine learning concepts, software security, communication with stakeholders, modern programming trends, and how to create portable, efficient, well-commented code.

Most notably, my other projects that didn't quite make the cut but were considered for my ePortfolio are the Gravioso Salvare project from CS340 which was about accessing a database of animals and locations, the pirate game machine learning project from CS370 which gets ever more relevant as AI becomes more and more popular, and the software testing project from CS 320 where we created a test class to document successful testing outcomes. Each of these projects was considered but ultimately, I felt that using the single project to highlight the skills I learned while working on each of these other artifacts would be much more appropriate.

Over the course of my studies, I have worked with C++, Java, Python, JavaScript/HTML/CSS, Matlab, Jupyter Notebook, MongoDB, SQL, and several other associated technologies to build projects. I have worked in Eclipse, PyCharm, VS Code, and with Git version control software. In my own time I've also built some small projects on my GitHub account that are not related to schoolwork. I have worked on front-end, back-end, and full-stack projects. I have worked with agile and waterfall development styles, done white and black box testing, and have some experience working with graphics libraries like OpenGL and Vulkan.

Since my project was one single artifact, I'll talk about how each of the pieces work together to make it work as a whole. To start, I'll focus on the code enhancement. Initially, this artifact was an inventory control app and a relatively simple one. All the user was able to do was utilize a rudimentary login feature to add items and quantities to a list which would display as a recycler view and then item quantities could be updated with a text message being sent to the phone when an item quantity reached 0. The artifact was enhanced, and its purpose shifted to that of a prescription refill tracking application with a more robust user account system that separates the data displayed by the currently logged in user. This involved many changes to the flow of data through the application and a lot of added comments and code improvements including improving the error handling with several new data types.

The database enhancement is represented as the improvements to storing items as prescriptions. Now when users want to update their prescription they can put in quantity but also days of supply and the date of pickup. This improves the original functionality where only quantity was considered and displayed but now users input the information, and the system displays the next day the prescription is available for refill. The item database now holds the name of the prescription, the number of tablets, the days of supply, and the date of pickup. The password storage has also been improved where passwords are now stored as hash values. The recycler view also treats each user's login as a separate session and only accesses the database to display medication associated with the currently logged in user.

The algorithm and data structure enhancement is the more complicated of the improvements to the project. This is a two-part enhancement with both the date storage and the functions that work to calculate the next refill date and the function to create the hashed password (and salt) and retrieve and compare hashes during login. This is done by creating a random salt and using a

256-bit hash algorithm several hundred times to convert the plain text into a hash value. The salt and hash are both stored as a string for comparison later. The string comparison doesn't decode the hash but instead retrieves the salt and hashed password, runs the entered password through the hashing function using the stored salt and compares the output. The date storage and refill date calculation is also done programmatically.